PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 13

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Link Github:

https://github.com/rezfahreza/PBF_2026.git

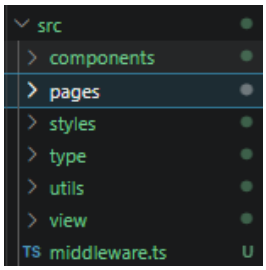
Praktikum

1. Membuat Middleware

- Modifikasi file index.tsx pada folder src/pages/produk

```
my-app > src > pages > produk > index.tsx > ...
import { useRouter } from "next/router";
import { useEffect, useState } from "react";
import useSWR from "swr";
import fetcher from "../../utils/swr/fetcher";
import TampilanProduk from "../../view/produk";
import { ProductType } from "@/type/product.type";

const Kategori = () => {
  const { push } = useRouter();
  const [products, setProducts] = useState([]);
  const { data, error, isLoading } = useSWR("/api/produk", fetcher);

  if (error) {
    return <p>Gagal mengambil data</p>;
  }

  return (
    <TampilanProduk products={isLoading ? [] : data.data} />
  );
};

export default Kategori;
```

- Buat file: src/middleware.ts

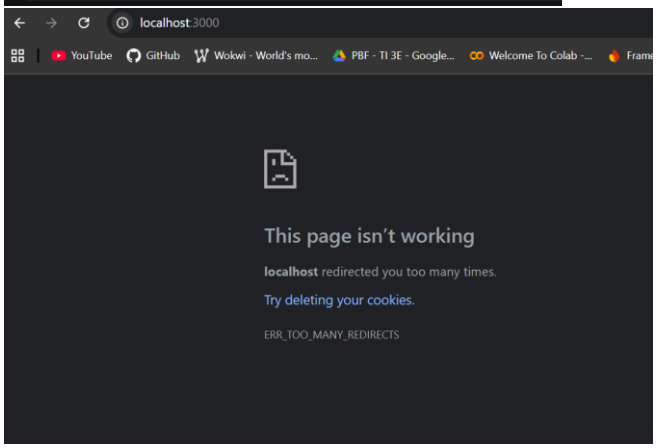


2. Struktur Dasar Middleware

```
my-app > src > middleware.ts > middleware
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    return NextResponse.next();
}
```

3. Redirect Sederhana

```
my-app > src > middleware.ts > middleware
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    return NextResponse.redirect(new URL("/", request.url));
}
```



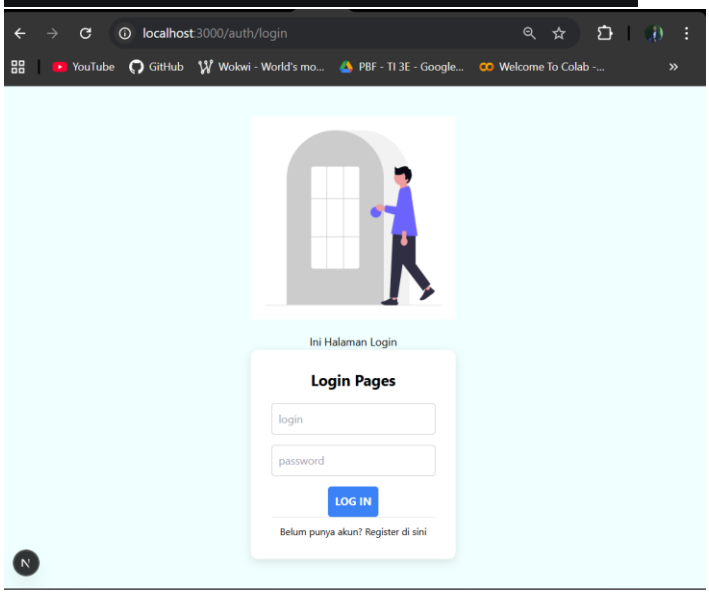
4. Batasi Route Tertentu

```
my-app > src > TS middleware.ts > config > matcher
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    return NextResponse.redirect(new URL("/", request.url));
}

export const config = {
  matcher: ["/produk", "/about"],
};
```

5. Simulasi Sistem Login

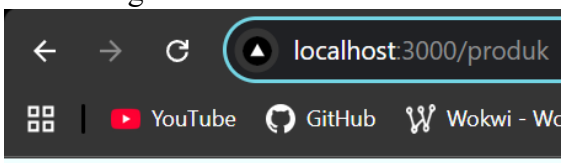
- Modifikasi file middleware.ts

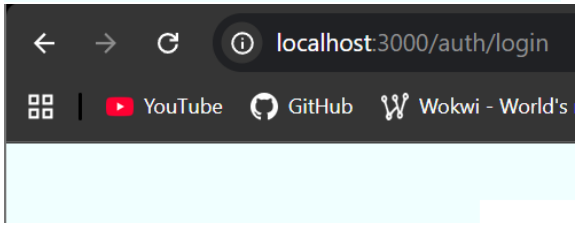
```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    const isLogin = false;
    if (isLogin) {
        return NextResponse.next();
    } else {
        return NextResponse.redirect(new URL("/auth/login", request.url));
    }
}

export const config = {
    matcher: ["/produk", "/about"],
};
```



6. Pengujian

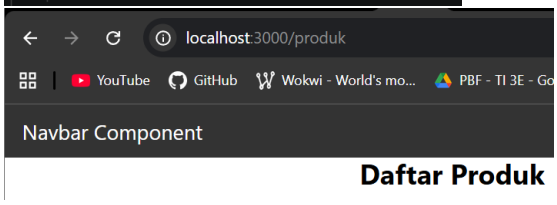
- isLogin = false





- isLogin = true

```
export function middleware(request: NextRequest) {
    const isLogin = true;
    if (isLogin) {
        return NextResponse.next();
```



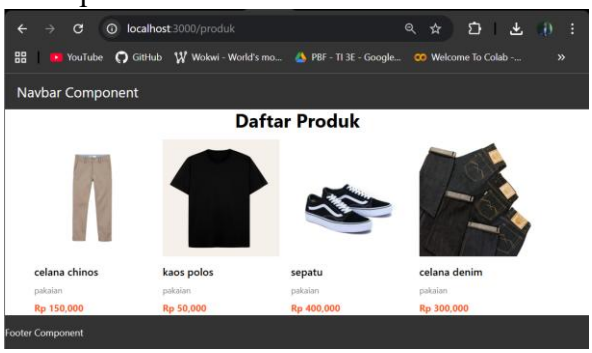
- Tambahkan Multiple Route

```
export const config = {
    matcher: ["/produk", "/about"],
};
```

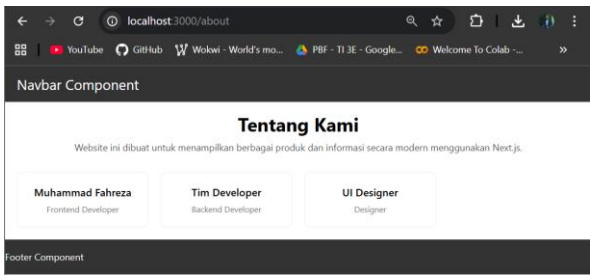
Tugas Praktikum

1. Buat halaman:

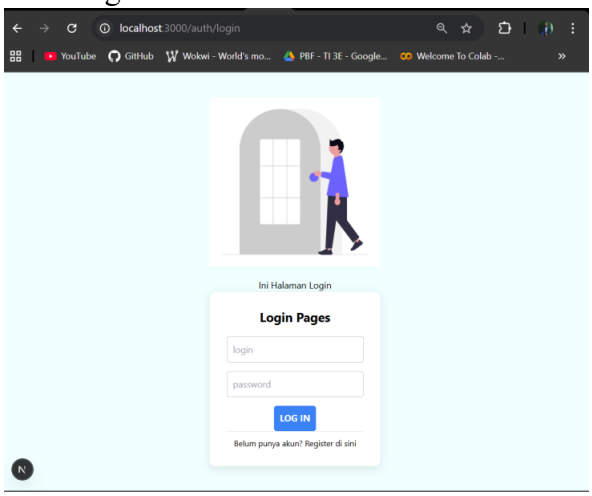
- /products



- /about



- /login



2. Implementasikan Middleware:

- Redirect ke /login jika belum login.

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    const isLogin = false;
    if (isLogin) {
        return NextResponse.next();
    } else {
        return NextResponse.redirect(new URL("/auth/login", request.url));
    }
}

export const config = {
    matcher: ["/produk", "/about"],
};
```

- Izinkan akses jika login true.

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function middleware(request: NextRequest) {
    const isLogin = true;
    if (isLogin) {
        return NextResponse.next();
    } else {
        return NextResponse.redirect(new URL("/auth/login", request.url));
    }
}

export const config = {
    matcher: ["/produk", "/about"],
};
```

3. Tambahkan proteksi hanya untuk route tertentu

```
export const config = {
    matcher: ["/produk", "/about"],
};
```

Pertanyaan Refleksi

1. Mengapa middleware lebih aman dibanding useEffect?

Jawab: Middleware lebih aman karena berjalan di server/edge layer sebelum halaman dirender. Hal ini mencegah pengguna melihat konten rahasia, berbeda dengan useEffect yang baru berjalan setelah halaman sempat terbuka di browser (client-side)

2. Mengapa middleware tidak menimbulkan glitch?

Jawab: Middleware tidak menimbulkan glitch karena proses pengalihan (redirect) dilakukan sebelum request halaman diproses dan sebelum konten dikirim ke browser. Sebaliknya, useEffect menyebabkan halaman "berkedip" karena konten asli sempat muncul sebentar sebelum akhirnya diarahkan ke halaman lain

3. Apa risiko jika semua halaman diproteksi tanpa pengecualian?

Jawab: Risiko utamanya adalah terjadinya error/infinite loading. Jika halaman /login juga ikut diproteksi, maka saat sistem mencoba mengarahkan pengguna ke halaman login, middleware akan kembali melakukan redirect, sehingga halaman tidak akan pernah bisa terbuka

4. Kapan middleware tidak diperlukan?

Jawab: Middleware tidak diperlukan pada halaman-halaman yang bersifat publik atau tidak membutuhkan pengecekan kondisi khusus (seperti status login atau role pengguna). Contohnya adalah halaman landing page atau dokumentasi publik yang bisa diakses siapa saja secara bebas

5. Apa perbedaan middleware dan API route?

Jawab:

- Middleware: Berfungsi sebagai "penghadang" (gatekeeper) yang mengecek dan memanipulasi request sebelum sampai ke tujuan untuk menentukan apakah request dilanjutkan atau diarahkan.
- API Route: Berfungsi sebagai "tujuan" (endpoint) itu sendiri untuk memproses permintaan data atau logika bisnis tertentu dari sisi server